



Technical Document

Skills Path Architecture

AUTOMATING THE DEVELOPMENT PHASE OF THE AGILE SDLC

Reliability · Consistency · Maintainability

Skills Path Architecture

How force-app/skills/development/ is shaped to **automate the Development phase of the Agile SDLC for the developer** while **adapting to this project's architecture and rules**, and how that shape resolves three concrete challenges: **reliability, consistency, maintainability**.

1. The goal

This skill set is not a grab-bag of recipes. It targets one intersection of the SDLC and one role, under one set of constraints.

Question	Answer
Which SDLC phase does this automate?	The Development phase of Agile.
Who uses it?	The developer .
For what?	To implement a task (the bug flow is reserved for later).
Constrained by what?	The project's architecture and rules — LWC architecture, Apex bulkification, soft-delete filter, required fields, controller → Service → APIResponse layering.

Restated:

Build a skill that **automates the Development phase of the Agile SDLC for the developer**, that **adapts to this project's architecture, rules, and framework**, and that delivers **reliability, consistency, and maintainability** without architectural overhead.

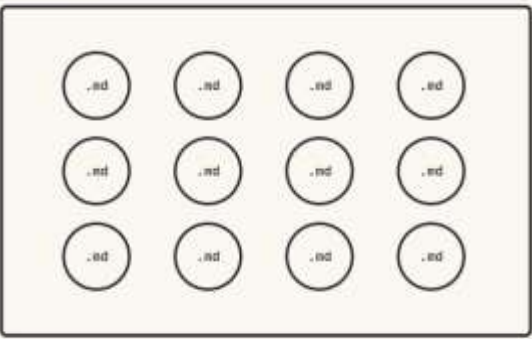
The rest of this document is the architecture that meets that goal, organized by the three challenges it has to solve.

2. From a flat pool to an orchestrator

The current shape is the third design in a progression — each step fixes the problem the previous one left open.

Solution 1 — One flat pool of skills

SOLUTION 1



The diagram shows a large rectangle containing a 3x4 grid of 12 small circles. Each circle contains the text ".md".

01 • One flat pool of skills

- Each circle is a single **.md** skill file, all sitting at the same level.
- From the user's prompt, the AI picks **one** file to load — but two skills can overlap on a specific point.
- To avoid grabbing the wrong one, the model must stay **aware of every skill's description at once**.

✗ **Doesn't scale.**

The more skills you add, the harder it is to keep all descriptions in mind — selection gets noisy and unreliable.

One flat pool of skills

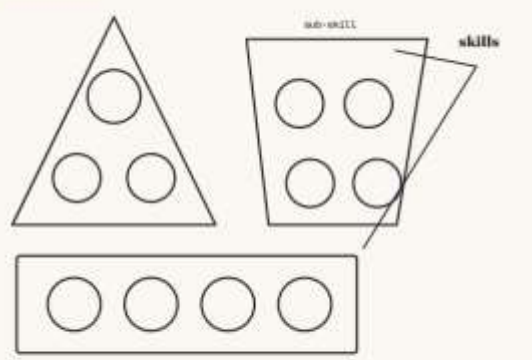
- Each circle is a single **.md** skill, all sitting at the same level.
- From the prompt the model picks **one** file — but two skills can overlap on a point.
- To avoid grabbing the wrong one, the developer must stay **aware of every skill's description at once**.

✗ **Doesn't scale.** Being aware of every skill is impossible once the system grows, and it costs more time on every prompt. The more skills you add, the noisier and less reliable selection gets.

Developer should be aware in all skill to avoid the conflict which is impossible and if it was it take along of time for check.

Solution 2 — Group into skills & sub-skills

SOLUTION 2



The diagram shows three shapes: a triangle, a trapezoid, and a rectangle. The triangle contains three circles, the trapezoid contains four circles, and the rectangle contains four circles. A line points from the label "sub-skill" to one of the circles in the trapezoid. Another line points from the label "skills" to the trapezoid itself.

02 • Group into skills & sub-skills

- Circles are bundled. Each **big shape is now one skill**; the small circles become **guides**, each with its own activation point — i.e. **sub-skills**.
- This adds structure: a prompt activates a sub-skill inside the right skill, instead of scanning a flat pile.
- But selection still happens **by meaning** — a semantic guess about which one fits.

~ **Better, but not enough.**

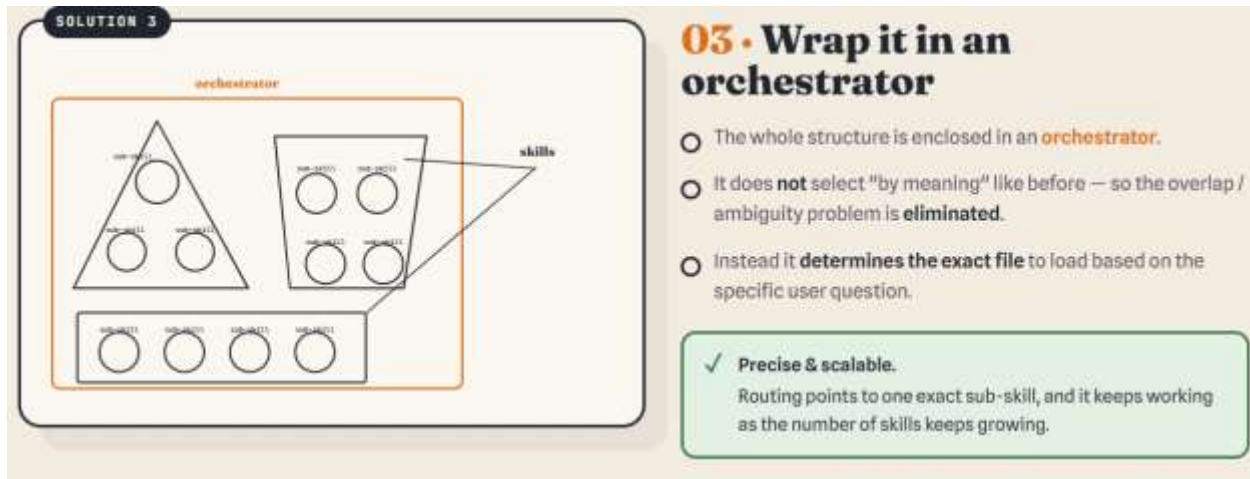
Grouping reduces the clutter, yet matching "by meaning" can still pick the wrong file when sub-skills are close.

Group into skills and sub-skills

- Circles are bundled: each big shape is one **skill**; the small circles become **guides** (sub-skills), each with its own activation point.
- A prompt activates a sub-skill inside the right skill instead of scanning a flat pile.

~ **Better, but not enough.** Selection still happens **by meaning** — a semantic guess — so it can still pick the wrong file when two sub-skills are close.

Solution 3 — Wrap it in an orchestrator



Wrap it in an orchestrator

- The whole structure is enclosed in an **orchestrator**.
- It does **not** select “by meaning”, so the overlap/ambiguity problem is eliminated.
- It determines the **exact** file to load from the specific user question.

✓ **Precise & scalable.** Routing points to one exact sub-skill, and it keeps working as the number of skills grows.

3. The three challenges (with examples)

2.1 Reliability — the right skill must fire even when the user didn't name it

Problem. A skill only fires if the model picks it. If the user's prompt doesn't name a downstream skill, it gets silently skipped.

Solution. Make the call **explicit inside the calling skill**, so it cannot be missed regardless of wording.

Example. The user says “create a new parent LWC” — nothing about CSS. The parent task-skill itself names the CSS handoff as a fixed step:

[create-new-parent-lwc-component.md:207](#) — Step 4 — Optionally apply lwc-css-design

2.2 Consistency — same intent, same path, every time

Problem. With N skills directly addressable, the user's wording decides which one fires. Same intent, different phrasings, different paths.

Solution. One entry point — the orchestrator — and an AskUserQuestion flow that turns the prompt into a fixed set of answers, so the user's intent is captured with 100% certainty.

Example. Any dev request enters [agile-development-orchestrator-skill.md](#) and answers two AskUserQuestion calls: bug vs. task, then A/B/C + parent/child. Three different phrasings of “*build a feature on a parent LWC*” all land on the same row:

B — create new functionality / parent → [add-interactive-with-data-persistence-functionality-in-pather.md](#)

2.3 Maintainability — a new task can't fan out edits across every other task

Problem. When several task-skills need the same sub-step, in-lining it in each one means every change has to touch every copy.

Solution. Extract the shared sub-step into one file under [task-skill/guide/](#) and have each task-skill call it.

Example. Both [add-interactive-with-data-persistence-functionality-in-pather.md](#) and [lwc-parent-event-handler-generator.md](#) need to resolve “*which Apex method backs this?*”. Instead of asking that interview manually in each file, both call [guide/apex-method-resolution-guide.md](#) — the three-branch interview (existing-known / existing-find / create-new) lives in exactly one file.

4. Adaptation to project architecture and rules

The orchestrator does not invent rules — it routes work into skills that already encode the project's rules. The chain of constraints, as they fire during a typical “create a new parent LWC” iteration:

1. **Orchestrator** — classifies as task / option A / parent.

2. [create-new-parent-lwc-component](#) — Step 0 mandates [lwc-architecture](#) end-to-end (page vs. child, principal state shape, localStorage keys, Apex methods, event names, sidecar utils / validator, meta.xml exposure).
3. **Per-iteration sub-skill** (A → functionality, B → event handler) runs its full disciplined interview, then hands off to the shared Apex-method-resolution protocol when controller methods need to be resolved.
4. **Final Verification** — the project-wide MANDATORY skills declared in [CLAUDE.md](#) are checked: bulkified Apex (apex-bulk-soql), soft-delete filter (soql-exclude-deleted), required fields per OBJECT_VALIDATION_LWC_APEX.md (object-required-fields), thin controller → Service → APIResponse layering (lwc-architecture).
5. **Optional CSS handoff** — Step 4 asks whether to apply [lwc-css-design](#) so the new parent drops in next to manageBacklog / manageWorkflow without visual tuning.

Every project-specific rule has exactly one home, and the orchestrator guarantees the rules are *reached* — not left to the model to remember.

5. Current file layout (for reference)

```
force-app/skills/development/
├── agile-development-orchestrator-skill.md    ← single entry
├── architecture/
│   ├── lwc-architecture.md
│   └── lwc-css-design.md
├── contract/
│   ├── object-required-fields-skill.md
│   └── soql-exclude-deleted-skill.md
├── guard/
│   └── apex-governor-limit-guard.md
├── performance/
│   ├── apex-bulk-soql.md
│   └── apex-method-monitor.md
├── task-skill/
│   ├── add-functionality-in-child-lwc-component.md
│   ├── add-interactive-with-data-persistence-functionality-in-parent.md
│   ├── create-new-child-lwc-component.md
│   ├── create-new-parent-lwc-component.md
│   ├── lwc-parent-event-handler-generator.md
│   └── guide/
│       ├── apex-method-resolution-guide.md
│       ├── lwc-request-loading-guide.md
│       └── lwc-error-handling-guide.md
```